

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

2-mashg'ulot. Socket moduli bilan ishlash.

O'quv savollari:

1. Socket modulining asosiy metodlari bilan tanishish.
2. Server yaratishda `.socket()`, `.bind()`, `.listen()`, `.accept()` metodlaridan foydalanish.
3. Klient yaratishda `.connect()`, `.send()`, `.recv()`, `.close()` metodlaridan foydalanish.
4. Klient va server o'rtasida ma'lumot almashishni amalga oshirish.

1. Socket modulining asosiy metodlari bilan tanishish.

Quyida Python `socket` moduli asosiy metodlari va ularning vazifalari keltirilgan:

Metod	Vazifasi
<code>socket()</code>	Yangi soket yaratadi. <code>socket.AF_INET</code> (IPv4) va <code>socket.SOCK_STREAM</code> (TCP) kabi parametrlar bilan ishlaydi.
<code>bind(address)</code>	Soketni muayyan manzil (IP manzil va port) bilan bog'laydi.
<code>listen(backlog)</code>	Soketni tinglash (yangi ulanishlarni kutish) holatiga o'tkazadi. <code>backlog</code> – navbat uzunligi.
<code>accept()</code>	Yangi ulanishni qabul qiladi va yangi soket (<code>conn</code>) bilan mijoz manzilini (<code>addr</code>) qaytaradi.
<code>connect(address)</code>	Klient soketini serverga bog'laydi (IP manzil va portga).
<code>send(data)</code>	Soket orqali ma'lumot yuboradi (qisman yuborilishi mumkin).
<code>sendall(data)</code>	Ma'lumotni to'liq yuboradi (qayta yuborilmagan bo'lsa, davom etadi).
<code>recv(bufsize)</code>	Soketdan ma'lumot qabul qiladi. <code>bufsize</code> – maksimal qabul qilish hajmi.
<code>close()</code>	Soketni yopadi, resurslarni bo'shatadi.
<code>settimeout(value)</code>	Soketning kutish vaqtini belgilaydi (sekundlarda). Soketga uzilish bo'lsa, xato qaytaradi.

2. Server yaratishda `.socket()`, `.bind()`, `.listen()`, `.accept()` metodlaridan foydalanish

1) `.socket()` — Soket yaratish

Vazifasi:

Yangi soket yaratadi. Bu metod server yoki klient uchun tarmoq ulanishi yaratishda asosiy vosita hisoblanadi.

Sintaksis:

```
socket.socket(family, type)
```

family: IP protokoli (masalan, `socket.AF_INET` — IPv4, `socket.AF_INET6` — IPv6)

type: Soket turi (masalan, `socket.SOCK_STREAM` — TCP, `socket.SOCK_DGRAM` — UDP)

Misol:

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print("Soket yaratildi!")
```

2) `.bind()` — Manzilni bog'lash

Vazifasi:

Server soketini muayyan IP manzil va port bilan bog'laydi. Server faqat ushbu manzildan keladigan ulanishlarni qabul qiladi.

Sintaksis:

```
socket.bind(address)
```

address: (host, port) — IP manzil va port tuple ko'rinishida.

Misol:

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Serverni localhost va port 5000 ga bog'lash
server_socket.bind(("127.0.0.1", 5000))
print("Soket manzilga bog'landi!")
```

3) `.listen()` — Ulanishlarni tinglash

Vazifasi:

Serverni ulanishlarni kutish holatiga o'tkazadi. `backlog` parametri yangi ulanishlar uchun navbat uzunligini belgilaydi.

Sintaksis:

```
socket.listen(backlog)
```

`backlog`: Maksimal ulanishlar soni (serverni kutayotgan ulanishlar navbati uzunligi). Misol uchun, `backlog=5` bu serverga 5 ta ulanishni kutish imkonini beradi.

Misol:

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))

# 5 ulanishni kutish
server_socket.listen(5)
print("Server ulanishlarni tinglamoqda...")
```

4) `.accept()` — Yangi ulanishni qabul qilish

Vazifasi:

Server yangi ulanishni qabul qiladi va u bilan ishlash uchun yangi soket (`conn`) va mijoz manzilini (`addr`) qaytaradi.

Sintaksis:

```
conn, addr = socket.accept()
```

`conn`: Mijoz bilan muloqot qilish uchun yangi soket.

`addr`: Mijozning manzili (IP manzil va port).

Misol:

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))
server_socket.listen(5)

print("Server ulanishlarni kutmoqda...")
conn, addr = server_socket.accept() # Yangi ulanishni kutib olish

print(f"Yangi mijoz ulanishi: {addr}")
```

Quyida `.socket()`, `.bind()`, `.listen()`, va `.accept()` metodlaridan foydalanib, oddiy server yaratishning to'liq misoli keltirilgan. Bu server mijozdan ma'lumotni qabul qilib, uni qaytaradi (echo server).

```
import socket

# 1) Soket yaratish
```

```

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) Soketni Localhost va port 5000 ga bog'lash
server_socket.bind(("127.0.0.1", 5000))

# 3) Serverni ulanishlarni kutish holatiga o'tkazish
server_socket.listen(5)
print("Server ulanishlarni kutmoqda...")

# 4) Yangi ulanishni qabul qilish
conn, addr = server_socket.accept() # Yangi ulanishni kutib olish
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish va qaytarish
while True:
    data = conn.recv(1024) # Mijozdan ma'lumot olish
    if not data:
        break # Ma'lumot bo'lmasa, chiqish
    print(f"Mijozdan keldi: {data.decode('utf-8')}")
    conn.sendall(data) # Qaytadan yuborish

conn.close() # Soketni yopish

```

Test qilish:

Serverni ishga tushurish:

Terminalda `python server.py` ni ishga tushiring.

Klientni ishga tushurish:

Yangi terminalda quyidagi kodni ishga tushiring:

```

import socket

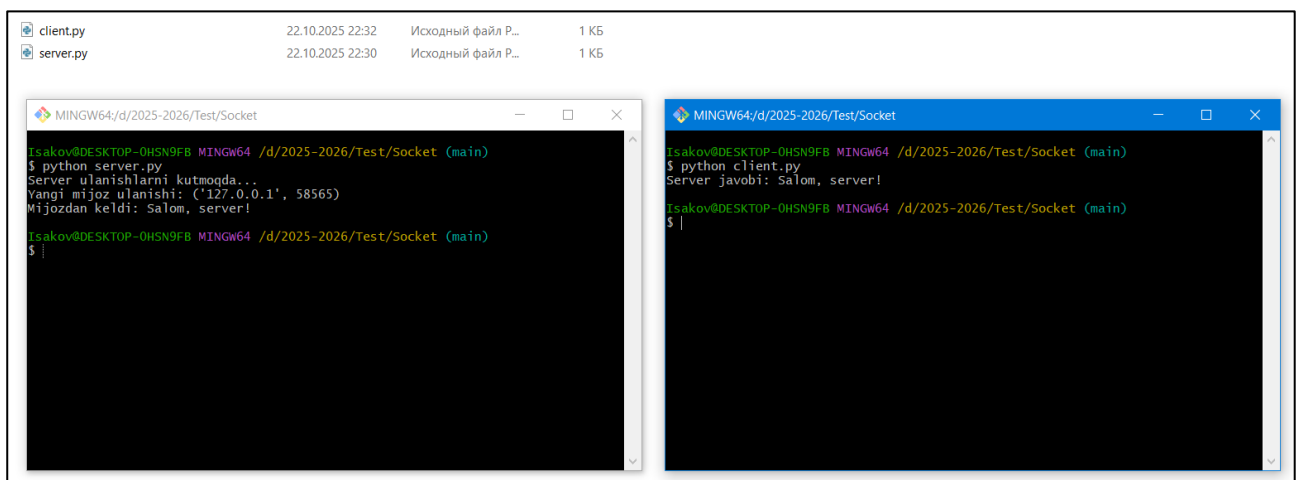
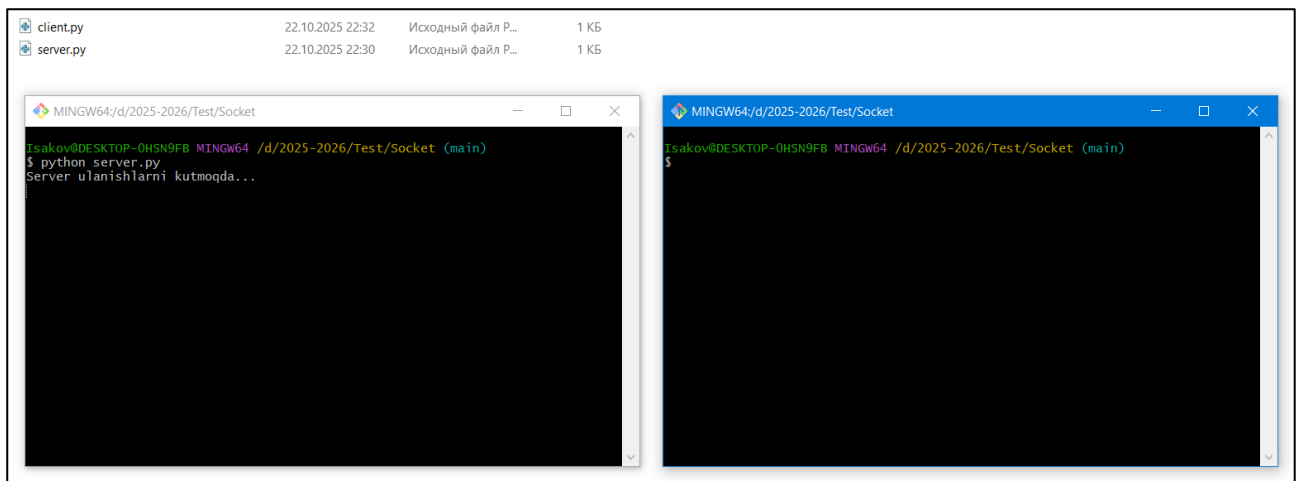
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

client_socket.sendall(b"Salom, server!")
response = client_socket.recv(1024)
print(f"Server javobi: {response.decode('utf-8')}")

client_socket.close()

```

Bu server mijozdan yuborilgan matnni qabul qilib, uni qaytaradi. Klient serverga "Salom, server!" xabarini yuboradi va server uni qaytaradi.



Bu holat shundan kelib chiqadi, chunki server faqat 5000 portida tinglayapti, ammo klient har safar ulanish uchun tizim tomonidan tasodifiy port tanlanadi. Shuning uchun, serverda 5000 porti ko'rsatilgan bo'lsa, klientning lokal porti har safar farq qiladi.

3. Klient yaratishda `.connect()`, `.send()`, `.recv()`, `.close()` metodlaridan foydalanish

1) `.connect()` — Serverga ulanish

Vazifasi:

Klient soketini serverga bog'lash uchun ishlatiladi. `connect()` metodi yordamida klient serverning IP manzili va portiga ulanadi.

Sintaksis:

```
socket.connect(address)
```

`address: (host, port)` — Server manzili va porti.

Misol:

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000)) # Serverning IP va portiga
ulanish
print("Serverga ulandi!")
```

2) .send() — Ma'lumot yuborish

Vazifasi:

Klient soketi orqali serverga ma'lumot yuboradi. `.send()` metodidan foydalanilganda, ba'zi ma'lumotlar yuborilmasligi mumkin, shuning uchun odatda `.sendall()` tavsiya etiladi, chunki bu barcha ma'lumot yuborilguncha ishlaydi.

Sintaksis:

```
socket.send(data)
```

`data`: Yuborilayotgan ma'lumot. `bytes` formatida bo'lishi kerak.

Misol:

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

client_socket.send(b"Salom, server!") # Serverga "Salom, server!" xabarini
yuborish
print("Xabar yuborildi!")
```

3) .recv() — Ma'lumot qabul qilish

Vazifasi:

Klient soketi orqali serverdan ma'lumot qabul qiladi. Bu metod **qaytarilgan baytlarni** (bytes) olish uchun ishlatiladi. `bufsize` parametri maksimal qabul qilinadigan baytlar sonini belgilaydi.

Sintaksis:

```
socket.recv(bufsize)
```

`bufsize`: Maksimal qabul qilish hajmi (masalan, 1024 bayt).

Misol:

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

client_socket.send(b"Salom, server!") # Xabar yuborish

response = client_socket.recv(1024) # Serverdan javob olish
print(f"Server javobi: {response.decode('utf-8')}")
```

4) `.close()` — Socketni yopish

Vazifasi:

Klient socketini yopish uchun ishlatiladi. Socketni yopish orqali, tarmoq resurslari boʻshatiladi va aloqani toʻxtatish mumkin.

Sintaksis:

```
socket.close()
```

Misol:

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

client_socket.send(b"Salom, server!") # Xabar yuborish
response = client_socket.recv(1024) # Serverdan javob olish
print(f"Server javobi: {response.decode('utf-8')}")

client_socket.close() # Socketni yopish
```

Quyida `.connect()`, `.send()`, `.recv()`, va `.close()` metodlaridan foydalanib, klientni yaratishning toʻliq misoli keltirilgan. Bu klient serverga maʼlumot yuboradi va javobni qaytarib oladi.

```
import socket

# 1) Soker yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) Serverga ulanish (serverning IP manzili va porti)
client_socket.connect(("127.0.0.1", 5000))

# 3) Ma'lumot yuborish
client_socket.send(b"Salom, server!") # Serverga "Salom, server!" yuboriladi

# 4) Serverdan javob olish
```

```
response = client_socket.recv(1024) # Serverdan 1024 bayt qabul qilinadi
print(f"Server javobi: {response.decode('utf-8')}")

# 5) Soketni yopish
client_socket.close()
```

Test qilish:

1. Serverni ishga tushurish:

- o **Server kodini** `server.py` faylida saqlang va `python server.py` buyrug'i bilan ishga tushiring.

2. Klientni ishga tushurish:

- o **Klient kodini** `client.py` faylida saqlang va `python client.py` buyrug'i bilan ishga tushiring.

3. Natija:

Klient "Salom, server!" xabarini yuboradi va serverdan xabarni qaytaradi.

Server terminalida:

```
Yangi mijoz ulanishi: ('127.0.0.1', 5001)
Mijozdan keldi: Salom, server!
```

Klient terminalida:

```
Server javobi: Salom, server!
```

Xulosa:

- `.connect()`: Klientni serverga bog'lash uchun ishlatiladi.
- `.send()`: Serverga ma'lumot yuboradi.
- `.recv()`: Serverdan ma'lumot qabul qiladi.
- `.close()`: Soketni yopadi va resurslarni bo'shatadi.

4. Klient va server o'rtasida ma'lumot almashishni amalga oshirish

Klient va server o'rtasida ma'lumot almashish — bu tarmoq dasturlari yaratishda eng muhim qism hisoblanadi. Klient ma'lumotni serverga yuboradi va server javob qaytaradi. Bu jarayonni amalga oshirishda `.send()`, `.recv()`, va `.close()` metodlaridan foydalaniladi.

Quyida, **klient-server** o'rtasida ma'lumot almashish jarayonini amalga oshirish uchun kerakli metodlar va misollarni keltiraman.

1) Klient tomonidan ma'lumot yuborish (`.send()`)

Vazifasi:

Klient serverga ma'lumot yuboradi. `.send()` metodi faqat ba'zi qismlarini yuborishi mumkin, shuning uchun ma'lumotni to'liq yuborish uchun `.sendall()` ishlatish afzal.

Sintaksis:

```
socket.send(data)
```

`data`: Yuboriladigan ma'lumot. `bytes` formatida bo'lishi kerak.

Misol:

```
import socket

# Klient socketini yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

# Serverga ma'lumot yuborish
message = "Salom, server!".encode("utf-8") # Matnni bytes formatiga
o'zgartirish
client_socket.sendall(message) # Ma'lumotni yuborish

client_socket.close() # Socketni yopish
```

2) Server tomonidan ma'lumot qabul qilish (`.recv()`)

Vazifasi:

Server klientdan kelgan ma'lumotni qabul qiladi. `.recv()` metodidan foydalaniladi, u yuborilgan baytlarni qaytaradi.

Sintaksis:

```
socket.recv(bufsize)
```

`bufsize`: Maksimal qabul qilinadigan baytlar soni. Masalan, 1024 — bu maksimal o'lchamdagi ma'lumot.

Misol:

```
import socket

# Server socketini yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))
server_socket.listen(5)

print("Server ulanishlarni kutmoqda...")
```

```

# Yangi ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish
data = conn.recv(1024) # Mijozdan ma'lumot olish
print(f"Mijozdan keldi: {data.decode('utf-8')}")

conn.close() # Soketni yopish

```

3) Server tomonidan ma'lumot yuborish (.send())

Vazifasi:

Server tomonidan ma'lumot yuborish. Server, klientga javob qaytarish uchun .send() metodidan foydalanadi.

Sintaksis:

```
socket.send(data)
```

data: Yuboriladigan ma'lumot. bytes formatida bo'lishi kerak.

Misol:

```

import socket

# Server soketini yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))
server_socket.listen(5)

print("Server ulanishlarni kutmoqda...")

# Yangi ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish
data = conn.recv(1024)
print(f"Mijozdan keldi: {data.decode('utf-8')}")

# Serverdan javob yuborish
response = "Xabar qabul qilindi".encode("utf-8")
conn.sendall(response) # Javob yuborish

conn.close() # Soketni yopish

```

4) Klient tomonidan ma'lumot qabul qilish (.recv())

Vazifasi:

Klient serverdan ma'lumot qabul qiladi. `.recv()` metodini ishlatib, serverdan yuborilgan javobni olish mumkin.

Sintaksis:

```
socket.recv(bufsize)
```

`bufsize`: Maksimal qabul qilinadigan baytlar soni.

Misol:

```
import socket

# Klient socketini yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

# Serverga ma'lumot yuborish
client_socket.sendall(b"Salom, server!")

# Serverdan javob olish
response = client_socket.recv(1024)
print(f"Server javobi: {response.decode('utf-8')}")

client_socket.close() # Socketni yopish
```

Quyida klient va server o'rtasida ma'lumot almashishni amalga oshiradigan to'liq misol keltirilgan. Klient serverga xabar yuboradi va server javob qaytaradi.

Server:

```
import socket

# Server socketini yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000)) # Serverni localhost va 5000-portga bog'lash
server_socket.listen(5)

print("Server ulanishlarni kutmoqda...")

# Yangi ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish
data = conn.recv(1024)
print(f"Mijozdan keldi: {data.decode('utf-8')}")
```

```
# Serverdan javob yuborish
response = "Xabar qabul qilindi".encode("utf-8")
conn.sendall(response) # Javob yuborish

conn.close() # Soketni yopish
```

Klient:

```
import socket

# Klient soketini yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000)) # Serverga ulanish

# Serverga ma'lumot yuborish
client_socket.sendall(b"Salom, server!")

# Serverdan javob olish
response = client_socket.recv(1024)
print(f"Server javobi: {response.decode('utf-8')}")

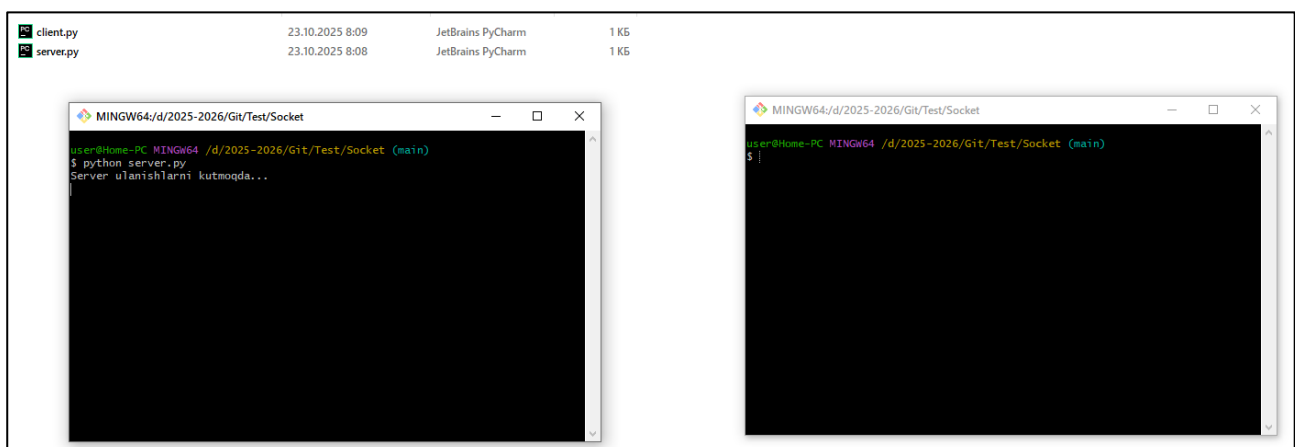
client_socket.close() # Soketni yopish
```

Test qilish:

1. Serverni ishga tushirish:

Server kodini `server.py` faylida saqlang va serverni ishga tushurish uchun terminalda:

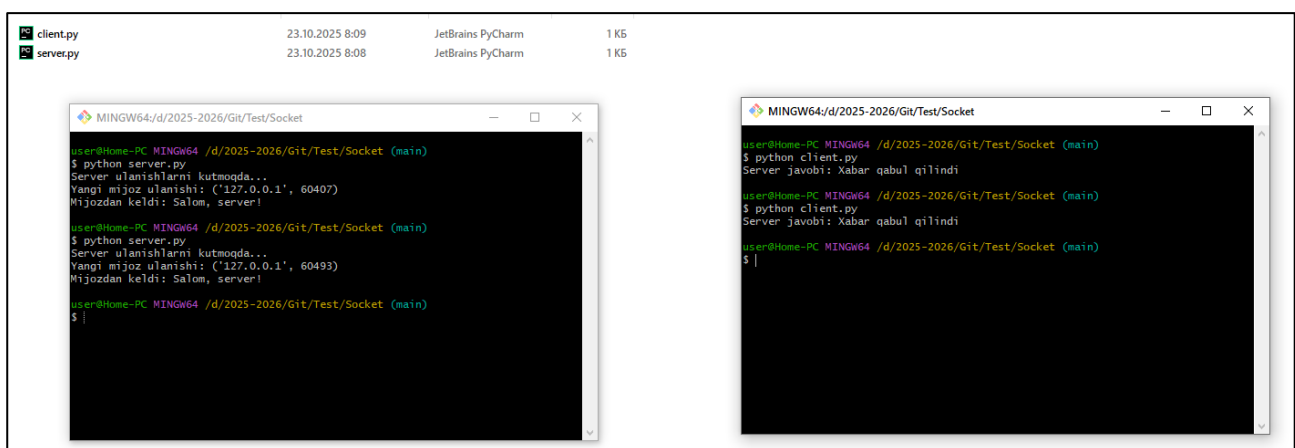
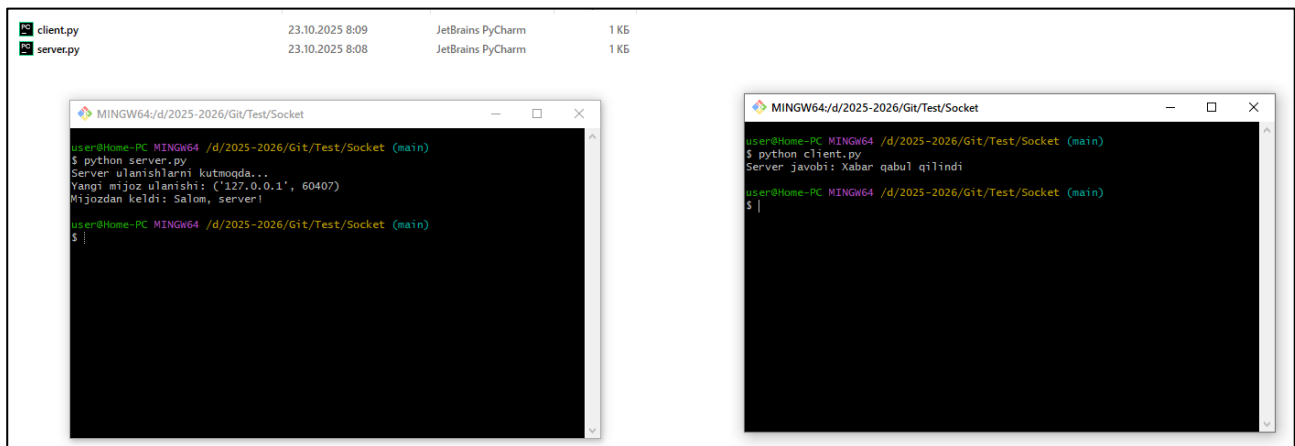
```
python server.py
```



2. Klientni ishga tushirish:

Klient kodini `client.py` faylida saqlang va klientni ishga tushurish uchun yangi terminalda:

python client.py



3. Natija:

Klient "Salom, server!" xabarini yuboradi va server javobini oladi.

Server terminalida:

```
Server ulanishlarni kutmoqda...
Yangi mijoz ulanishi: ('127.0.0.1', tasodifiy_port_raqami)
Mijozdan keldi: Salom, server!
```

Bu holat shundan kelib chiqadi, chunki server faqat 5000 portida tinglayapti, ammo klient har safar ulanish uchun tizim tomonidan tasodifiy port tanlanadi. Shuning uchun, serverda 5000 porti ko'rsatilgan bo'lsa, klientning lokal porti har safar farq qiladi.

Klient terminalida:

```
Server javobi: Xabar qabul qilindi
```

Xulosa:

- **Klient:** Ma'lumot yuborish uchun `.send()` va javob olish uchun `.recv()` metodlarini ishlatadi.
- **Server:** Ma'lumot qabul qilish uchun `.recv()` va javob yuborish uchun `.send()` metodlarini ishlatadi.
- `.sendall()`: Ma'lumotning to'liq yuborilishini ta'minlash uchun ishlatiladi.
- `.close()`: Soketni yopish uchun ishlatiladi, bu resurslarni bo'shatadi.